Replacing Mistral with Qwen 3.5: llama-server Setup Guide

Problem

Nanobot currently runs Mistral Small 3.2 24B on Ollama because Qwen 3.5's tool calling is broken in Ollama. Mistral handles tool calls fine but is weak at code evaluation — Qwen 3.5 27B is strong at both, but Ollama has multiple bugs in its Qwen 3.5 template renderer that break tool calling.

Running Qwen 3.5 through llama.cpp's `llama-server` with a community-fixed chat template fixes all tool calling issues, letting Qwen 3.5 27B replace Mistral entirely.

What's broken in Ollama

- Wrong tool calling pipeline (Hermes JSON instead of Coder XML format)
- Unclosed `<think>` blocks when tool calls follow thinking
- Missing generation prompt after tool call turns
- `presence_penalty` silently discarded on the Go runner

These are Ollama bugs, not model bugs.

Architecture

```
NanobotVM (10.0.0.163)
  └── Nanobot service → http://10.0.0.132:8080/v1/chat/completions

MAYU-PC (10.0.0.132)
  ├── Docker: ollama        → port 11434 (retire or keep for other models)
  └── Docker: llama-server  → port 8080  (Qwen 3.5 27B, tool calling works)
```

VRAM Budget (RTX 3090 — 24GB)

Qwen 3.5 27B Q4_K_M requires careful VRAM planning. KV cache quantization (`q8_0`) roughly halves cache size, which is what makes 49K context feasible.

Component	Without KV quant	With `--cache-type-k q8_0 --cache-type-v q8_0`
Model weights (Q4_K_M)	~17 GB	~17 GB
KV cache @ 32K ctx	~4–5 GB	~2–3 GB
KV cache @ 49K ctx	~6–7 GB (tight)	~3–4 GB
KV cache @ 64K ctx	~9–10 GB (won't fit)	~5–6 GB (tight)
Total @ 49K ctx	**~23–24 GB (risky)**	**~20–21 GB (comfortable)**

Nanobot context usage per request

Component	Tokens
AGENTS.md (loaded every request)	~6–8K
MCP tool schemas (~20 tools)	~2–3K
Conversation history (memoryWindow turns)	Variable
Model response (maxTokens: 16384)	Up to 16K

At memoryWindow: 100 — a full session hits ~35–45K input + 16K response = 51–61K total. This exceeds the 49K context window and will fail on deep sessions.

At memoryWindow: 30 — a full session hits ~15–20K input + 16K response = 31–36K total. Fits in 49K comfortably with headroom.

The model doesn't need 100 turns of raw history for Socratic tutoring. The last 20–30 exchanges maintain full coaching context. Problem state and curriculum progress are tracked by tools, not conversation history.

Step 1: Install hf CLI (via WSL)

`hf` is the HuggingFace Hub CLI for downloading models. It handles large files, resumable downloads, and verifies integrity — much better than browser downloads for 17 GB files.

Run all download steps from WSL. The files are written to `/mnt/c/` which maps directly to the Windows filesystem, so Docker can mount them without any copying.

Install via pipx

```zsh
sudo apt install pipx
pipx install 'huggingface_hub[cli]'
pipx inject huggingface_hub hf_transfer
```

Enable hf_transfer for faster downloads

```zsh
echo 'export HF_HUB_ENABLE_HF_TRANSFER=1' >> ~/.zshrc
source ~/.zshrc
```

Verify

```zsh
hf version
```

No HuggingFace account is needed — these are public repos.

Step 2: Create the Model Directory

From WSL:

```zsh
mkdir -p /mnt/c/Users/U/Documents/llama-cpp/models/qwen3.5-48k-q4-attuned
```

Step 3: Download the Model GGUF

Download the Unsloth Q4_K_M quant (~17 GB). This uses improved quantization with imatrix data and fixed chat templates:

```zsh
hf download unsloth/Qwen3.5-27B-GGUF Qwen3.5-27B-Q4_K_M.gguf --local-dir /mnt/c/User
```

If the download is interrupted, just rerun the same command — it resumes where it left off.

Step 4: Download the Fixed Chat Template

The community-fixed template by barubary (21 fixes for tool calling, thinking mode, parallel tool calls):

```zsh
hf download barubary/qwen3.5-barubary-attuned-chat-template chat_template.jinja --lo
```

Verify both files

```zsh
ls -lh /mnt/c/Users/U/Documents/llama-cpp/models/qwen3.5-48k-q4-attuned/
# Qwen3.5-27B-Q4_K_M.gguf    (~17 GB)
# chat_template.jinja          (~few KB)
```

Step 5: Stop Ollama, Start llama-server

```powershell
# Free the GPU
docker stop ollama

# Start llama-server
docker run -d --name llama-server ^
  --gpus all ^
  -p 8080:8080 ^
  -v C:\Users\U\Documents\llama-cpp\models\qwen3.5-48k-q4-attuned:/models ^
  ghcr.io/ggml-org/llama.cpp:server-cuda ^
  --model /models/Qwen3.5-27B-Q4_K_M.gguf ^
  --jinja ^
  --chat-template-file /models/chat_template.jinja ^
  --host 0.0.0.0 ^
  --port 8080 ^
  --n-gpu-layers 99 ^
  --ctx-size 49152 ^
  --cache-type-k q8_0 ^
  --cache-type-v q8_0 ^
  --presence-penalty 1.5
```

Parameter Notes

Parameter	Value	Notes
`--n-gpu-layers 99`	All layers on GPU	3090 24GB fits 27B Q4_K_M
`--ctx-size 49152`	49K context	Matches Mistral 48K setup. Requires KV cache quant
`--cache-type-k q8_0`	Quantize KV cache keys	Halves KV cache VRAM. Minimal quality impact
`--cache-type-v q8_0`	Quantize KV cache values	Same as above for values
`--presence-penalty 1.5`	Qwen 3.5 recommended	Prevents repetition loops in thinking
`--jinja`	Required	Enables the fixed community template

Verify

```powershell
# Health check
curl http://localhost:8080/health

# Quick chat test
curl -s http://localhost:8080/v1/chat/completions ^
  -H "Content-Type: application/json" ^
  -d "{\"messages\":[{\"role\":\"user\",\"content\":\"hello\"}],\"max_tokens\":50}"

# Tool calling test
curl -s http://localhost:8080/v1/chat/completions ^
  -H "Content-Type: application/json" ^
  -d "{\"messages\":[{\"role\":\"user\",\"content\":\"What is the weather in Seattle
```

The tool calling test should return a proper `tool_calls` array in the response, not raw XML text.

Step 6: Update Nanobot

Endpoint Change

```
Old: POST http://10.0.0.132:11434/api/chat          (Ollama)
New: POST http://10.0.0.132:8080/v1/chat/completions (llama-server)
```

Response Parsing Change

This is the only code change:

```javascript
// OLD — Ollama format
const content = response.message.content;
const toolCalls = response.message.tool_calls;

// NEW — OpenAI format
const content = response.choices[0].message.content;
const toolCalls = response.choices[0].message.tool_calls;
```

Nanobot Config Change — memoryWindow

Reduce `memoryWindow` from 100 to 30:

```javascript
// OLD
memoryWindow: 100

// NEW
memoryWindow: 30
```

This keeps full session context at ~31-36K tokens, well within the 49K context window. The model maintains full coaching quality with 30 turns of history — problem state and curriculum progress are tracked by tools, not raw conversation replay.

Tool Call Response Format

Tool calls come back in standard OpenAI format:

```json
```

```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": null,
        "tool_calls": [
          {
            "id": "call_abc123",
            "type": "function",
            "function": {
              "name": "get_weather",
              "arguments": "{\"city\": \"Seattle\"}"
            }
          }
        ]
      }
    }
  ]
}
```

Sending Tool Results Back

When returning tool results to the model, use role `"tool"` with a matching `tool_call_id`:

```
{
  "messages": [
    {"role": "user", "content": "What's the weather in Seattle?"},
    {"role": "assistant", "content": null, "tool_calls": [{"id": "call_abc123", "typ
    {"role": "tool", "tool_call_id": "call_abc123", "content": "{\"temp\": 55, \"con
  ]
}
```

Streaming Difference (if applicable)

If Nanobot uses `"stream": true`, the format changes:

Ollama — newline-delimited JSON:

```
{"message":{"content":"Hello"},"done":false}
{"message":{"content":""},"done":true}
```

llama-server — Server-Sent Events:

```
data: {"choices":[{"delta":{"content":"Hello"}}]}
data: [DONE]
```

If not streaming, ignore this.

Auto-Start on Boot

If Ollama was set to auto-start, update to start llama-server instead:

```powershell
# Set llama-server to auto-restart
docker update --restart unless-stopped llama-server

# Remove auto-restart from Ollama
docker update --restart no ollama
```

Fallback: Drop to 9B if 27B is too tight

If 27B hits VRAM limits or feels too slow, swap to 9B without changing anything in Nanobot:

```zsh
# From WSL — download 9B
mkdir -p /mnt/c/Users/U/Documents/llama-cpp/models/qwen3.5-9b-48k-q4-attuned
hf download unsloth/Qwen3.5-9B-GGUF Qwen3.5-9B-Q4_K_M.gguf --local-dir /mnt/c/Users/
hf download barubary/qwen3.5-barubary-attuned-chat-template chat_template.jinja --lo
```

```powershell
```

```
# From PowerShell — recreate container with 9B
docker stop llama-server && docker rm llama-server

docker run -d --name llama-server ^
  --gpus all ^
  -p 8080:8080 ^
  -v C:\Users\U\Documents\llama-cpp\models\qwen3.5-9b-48k-q4-attuned:/models ^
  ghcr.io/ggml-org/llama.cpp:server-cuda ^
  --model /models/Qwen3.5-9B-Q4_K_M.gguf ^
  --jinja ^
  --chat-template-file /models/chat_template.jinja ^
  --host 0.0.0.0 ^
  --port 8080 ^
  --n-gpu-layers 99 ^
  --ctx-size 49152 ^
  --cache-type-k q8_0 ^
  --cache-type-v q8_0 ^
  --presence-penalty 1.5
```

9B Q4_K_M uses ~6.6 GB for weights, leaving ~17 GB for KV cache — 49K context fits trivially. Same API, same endpoint, Nanobot doesn't know the difference.

Troubleshooting

Issue	Fix
Container won't start / OOM	Reduce `--ctx-size` to 32768. Confirm Ollama is stopped (`nvidia-smi`)
OOM during long sessions	Reduce Nanobot `memoryWindow` further (try 20). Or drop `--ctx-size` to 32768
Tool calls returned as plain text	Verify `--jinja` and `--chat-template-file` flags. Check `docker logs llama-server`
Can't reach from NanobotVM	Confirm `--host 0.0.0.0`. Check Windows firewall allows port 8080. Test: `curl http://10.0.0.132:8080/health`
Slow first response	Normal — first request loads model into VRAM. Subsequent requests are fast
Slow inference vs Mistral	Try 9B (see Fallback above). Or reduce `--ctx-size` to lower KV cache overhead

Issue	Fix
Repetition loops in thinking	Confirm `--presence-penalty 1.5` is set
Want to go back to Mistral/Ollama	`docker stop llama-server && docker start ollama`

Summary

	Before	After
Model	Mistral Small 3.2 24B (Ollama)	Qwen 3.5 27B (llama-server)
Model path	Ollama managed	`C:\Users\U\Documents\llama-cpp\models\qwen3.5-48k-q4-attuned`
Port	11434	8080
Context window	48K	49K
Endpoint	`/api/chat`	`/v1/chat/completions`
Response path	`response.message.content`	`response.choices[0].message.content`
Tool calling	Working (Mistral)	Working (Qwen 3.5 + fixed template)
Code evaluation	Weak	Strong
memoryWindow	100	30 (recommended)
KV cache	Default (FP16)	Quantized (q8_0) — saves ~3 GB VRAM
Code change in Nanobot	—	Endpoint URL + response parsing + memoryWindow